Python OOP Assignment

Classes & Objects · Inheritance · Encapsulation · Polymorphism

Topics	Classes & Objects · Inheritance · Encapsulation · Polymorphism
Difficulty	10 Easy · 10 Medium
Total	20 questions

Section 1 — Easy Q1 – Q10

Each question focuses on one OOP concept at a time — building classes, single-level inheritance, private attributes, and basic method overriding.

Q1 Create a basic Car class

EASY | Class & Object

Create a Car class with __init__ that takes brand and model. Add a method display_info() that prints both. Create two Car objects and call the method on each.

INPUT

```
car1 = Car("Toyota", "Corolla")
car2 = Car("Honda", "Civic")
car1.display_info()
car2.display_info()
```

EXPECTED OUTPUT

```
Brand: Toyota, Model: Corolla
Brand: Honda, Model: Civic
```

Q2 Track number of objects created

EASY | Class & Object

Create a class Counter with a class variable count starting at 0. Every time a new object is created, increase count by 1. Add a class method get_count() that returns the current count.

INPUT

```
a = Counter()
b = Counter()
c = Counter()
print(Counter.get_count())
```

EXPECTED OUTPUT

```
3
```

Q3 Rectangle area and perimeter

EASY | Class & Object

Create a Rectangle class that takes length and width. Add methods area() and perimeter() that return the respective calculations.

INPUT

```
r = Rectangle(5, 3)
print(r.area())
print(r.perimeter())
```

EXPECTED OUTPUT

```
15
16
```

Q4 Protect balance with a private attribute

EASY | Encapsulation

Create a BankAccount class with a private attribute __balance set in __init__. Add deposit(amount) and get_balance() methods. Do not allow direct access to __balance from outside the class.

INPUT

```
acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())
```

EXPECTED OUTPUT

```
1500
```

Q5 Animal and Dog inheritance

EASY | Inheritance

Create a base class Animal with a method speak() that returns 'Some sound'. Create a Dog class that inherits from Animal and overrides speak() to return 'Bark'. Create one object of each and call speak().

INPUT

```
a = Animal()
d = Dog()
```

EXPECTED OUTPUT

```
Some sound
Bark
```

```
print(a.speak())
print(d.speak())
```

Q6 Student class with default values

EASY | Class & Object

Create a Student class where age defaults to 18 if not provided. The __init__ should accept name and an optional age parameter.

INPUT

```
s1 = Student("Ali")
s2 = Student("Mia", 21)
print(s1.name, s1.age)
print(s2.name, s2.age)
```

EXPECTED OUTPUT

```
Ali 18
Mia 21
```

Q7 Use a property to validate age

EASY | Encapsulation

Create a Person class with a private attribute __age. Use the @property decorator to create a getter for age, and a setter that raises a ValueError if the age is negative.

INPUT

```
p = Person("Sam", 25)
print(p.age)
p.age = -5
```

EXPECTED OUTPUT

```
25
ValueError: Age cannot be negative
```

Q8 Use super() to extend the parent constructor

EASY | Inheritance

Create a Vehicle class with __init__ that sets brand. Create a Car class that inherits from Vehicle, adds a model attribute, and calls super().__init__() to set the brand.

INPUT	EXPECTED OUTPUT
`c = Car("Toyota", "Corolla")` `print(c.brand, c.model)`	`Toyota Corolla`

Q9 Same method name, different shapes

EASY | Polymorphism

Create a Shape base class with a method area() returning 0. Create Circle and Square subclasses that each override area() with their own formula. Loop through a list of shape objects and call area() on each.

INPUT	EXPECTED OUTPUT

`shapes = [Circle(5), Square(4)]`
`for s in shapes:`
`  print(s.area())`
	`78.5`
	`16`

Q10 String representation using __str__

EASY | Class & Object

Create a Book class with title and author. Override the __str__ method so that printing a Book object shows a formatted string instead of the default object representation.

INPUT	EXPECTED OUTPUT
`b = Book("Dune", "Frank Herbert")` `print(b)`	`'Dune' by Frank Herbert`

Section 2 — Medium Q11 – Q20

These questions combine multiple concepts — multi-level inheritance, super() calls inside overridden methods, validation logic, and polymorphism with or without a common parent class.

Q11 Three-level inheritance chain

MEDIUM | Inheritance

Create a class Vehicle with attribute wheels. Create Car that inherits from Vehicle and adds brand. Create SportsCar that inherits from Car and adds top_speed. Create a SportsCar object and print all three attributes.

INPUT

```
sc = SportsCar(4, "Ferrari", 340)
print(sc.wheels, sc.brand,
sc.top_speed)
```

EXPECTED OUTPUT

```
4 Ferrari 340
```

Q12 Method overriding with super() call

MEDIUM | Polymorphism

Create an Employee class with a method calculate_bonus() returning 1000. Create a Manager subclass that overrides calculate_bonus() to call the parent's value using super() and adds an extra 2000 on top.

INPUT

```
e = Employee()
m = Manager()
print(e.calculate_bonus())
print(m.calculate_bonus())
```

EXPECTED OUTPUT

```
1000
3000
```

Q13 Inventory system with private stock

MEDIUM | Encapsulation

Create a Product class with a private __stock attribute. Add methods sell(quantity) that reduces stock only if enough is available (otherwise print an error), and restock(quantity) that increases it. Add a method check_stock() to view current stock.

INPUT

```
p = Product("Pen", 50)
p.sell(20)
p.sell(40)
p.restock(15)
```

EXPECTED OUTPUT

```
Not enough stock!
45
```

```
print(p.check_stock())
```

Q14 Multiple subclasses sharing one parent method

MEDIUM | Inheritance

Create a base class Employee with __init__ for name and base_salary, and a method total_pay() returning base_salary. Create Developer and Designer subclasses, each overriding total_pay() to add a different fixed bonus on top of the base.

INPUT

```
d = Developer("Ali", 50000)
de = Designer("Mia", 45000)
print(d.total_pay())
print(de.total_pay())
```

EXPECTED OUTPUT

```
55000
48000
```

Q15 Polymorphism with a common interface

MEDIUM | Polymorphism

Create a PaymentMethod base class with a method pay(amount) that raises NotImplementedError. Create CreditCard and PayPal subclasses that each implement pay() differently (different printed message). Loop through a list of payment objects and call pay() on each with the same amount.

INPUT

```
methods = [CreditCard(), PayPal()]
for m in methods:
    m.pay(100)
```

EXPECTED OUTPUT

```
Paid $100 using Credit Card
Paid $100 using PayPal
```

Q16 Read-only property derived from private data

MEDIUM | Encapsulation

Create a Circle class with a private __radius set in __init__. Add a read-only property called area that calculates and returns the area each time it is accessed — without storing it. Do not allow area to be set directly.

INPUT

```
c = Circle(10)
print(c.area)
c.area = 500
```

EXPECTED OUTPUT

```
314.0
AttributeError: can't set attribute
```

Q17 Override and extend a method's behaviour

MEDIUM | Inheritance

Create a Shape class with a method describe() that returns 'This is a shape'. Create a Triangle subclass that overrides describe() to call the parent's describe() using super() and appends ' - specifically a triangle' to the result.

INPUT

```
t = Triangle()
print(t.describe())
```

EXPECTED OUTPUT

```
This is a shape - specifically a triangle
```

Q18 Compare two objects using a custom method

MEDIUM | Class & Object

Create a Student class with name and score. Add a method called compare(other) that returns the name of whichever student (self or other) has the higher score. Use this method to compare two student objects.

INPUT

```
s1 = Student("Ali", 85)
s2 = Student("Mia", 92)
print(s1.compare(s2))
```

EXPECTED OUTPUT

```
Mia
```

Q19 Duck typing without inheritance

MEDIUM | Polymorphism

Create two unrelated classes, Duck and Robot, each with their own make_sound() method returning different strings. Without using a common parent class, loop through a list containing one of each and call make_sound() on every object — demonstrating that Python does not require inheritance for polymorphism.

INPUT

```
objects = [Duck(), Robot()]
for obj in objects:
    print(obj.make_sound())
```

EXPECTED OUTPUT

```
Quack!
Beep Boop!
```

Q20 Build a class with full encapsulation and validation

MEDIUM | Encapsulation

Create a Temperature class storing a private __celsius value. Provide a property celsius with a getter and a setter that raises a ValueError if the value is below -273.15 (absolute zero). Also add a read-only property fahrenheit that converts and returns the value derived from celsius.

INPUT

```
t = Temperature(25)
print(t.celsius)
print(t.fahrenheit)
t.celsius = -300
```

EXPECTED OUTPUT

```
25
77.0
ValueError: Temperature below
absolute zero
```